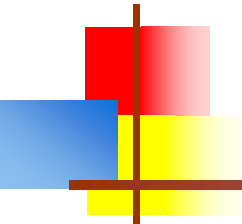


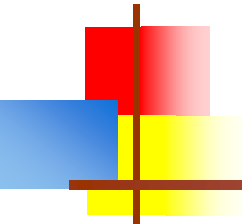
HTML





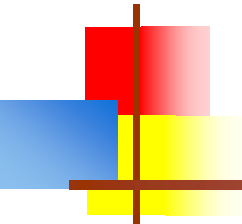
What is HTML?

- HTML stands for **Hypertext Markup Language**
- An HTML file is a text file containing **markup tags**
 - The markup tags tell the Web browser how to display the page
- HTML files must have an **htm** or **html** file extension
 - **.html** is preferred
 - **.htm** is from very old operating systems that can only handle “8+3” names (eight characters, dot, three characters)
- HTML files can be created using a simple text editor



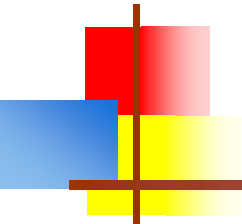
History of HTML

- Invented by Tim Berners-Lee, first web page published August 6, 1991
- Created "hypertext" to share scientific papers
- Intended as a standard way to structuring documents
- Standardized by [W3C](#) (World Wide Web Consortium - pack of super nerds)
- Early 90s
- HTML 4 in 1997
- XHTML in 2000
- HTML 5 in 2014



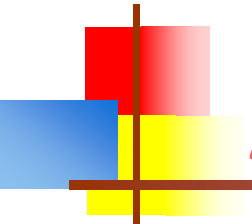
HTML Tags

- HTML tags are used to mark up HTML **elements**
- HTML tags are surrounded by **angle brackets**, `<` and `>`
- Most HTML tags come in pairs, like `<b>` and `</b>`
- The tags in a pair are the **start tag** and the **end tag**
- The text between the start and end tags is the **element content**
- The tags act as containers (they contain the element content), and should be properly nested
- HTML tags are **not** case sensitive; `<b>` means the same as `<B>`



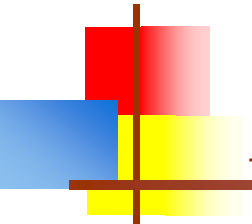
Terms

- **Web design:** The process of planning, structuring and creating a website
- **Web development:** The process of programming dynamic web applications
- **Front end:** The outwardly visible elements of a website or application
- **Back end:** The inner workings and functionality of a website or application.
- **Client & Server:**
 - Web Servers: IIS, Apache HTTP Server, NGINX, Lighttpd
 - Browsers: Chrome, Firefox, etc.



Anatomy of a Website

- **Your Content**
 - + **HTML**: Structure
 - + **CSS**: Presentation
 - = **Your Website**
- A website is a way to present your content to the world, using HTML and CSS to present that content & make it look good.



Anatomy of a Website

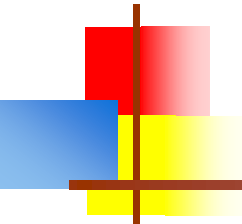
Concrete example:

- A paragraph is your **content**
- Putting your content into a `<p>` HTML tag to indicate it is a paragraph is creating **structure**

`<p>`A paragraph is your content`</p>`

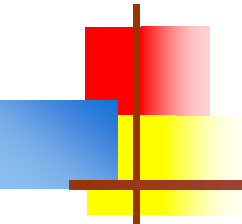
- Making the font of your paragraph green and 24px is **presentation**

A paragraph is your content



Some simple tags

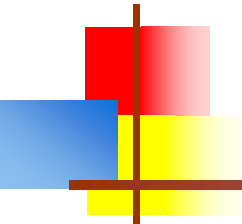
- `<!doctype ..>` Identifies the type of HTML doc
- `<html>` Marks the start and end of HTML document
- `<head>` Beginning and ending of Head section
- `<body>` Represents start and end of body of HTML doc
- `<title>` Represents the title that is displayed on title bar
- `<h1>` through `<h6>` : Headers, from largest to smallest
- `<b>` Boldface
- `<i>` Italic
- `<p>` Paragraph
- `<pre>` Preformatted text; preserve spaces and don't wrap lines
- `<br>` Inserts a line break
- `<!-- -->` Defines a comment that is ignored by the browser



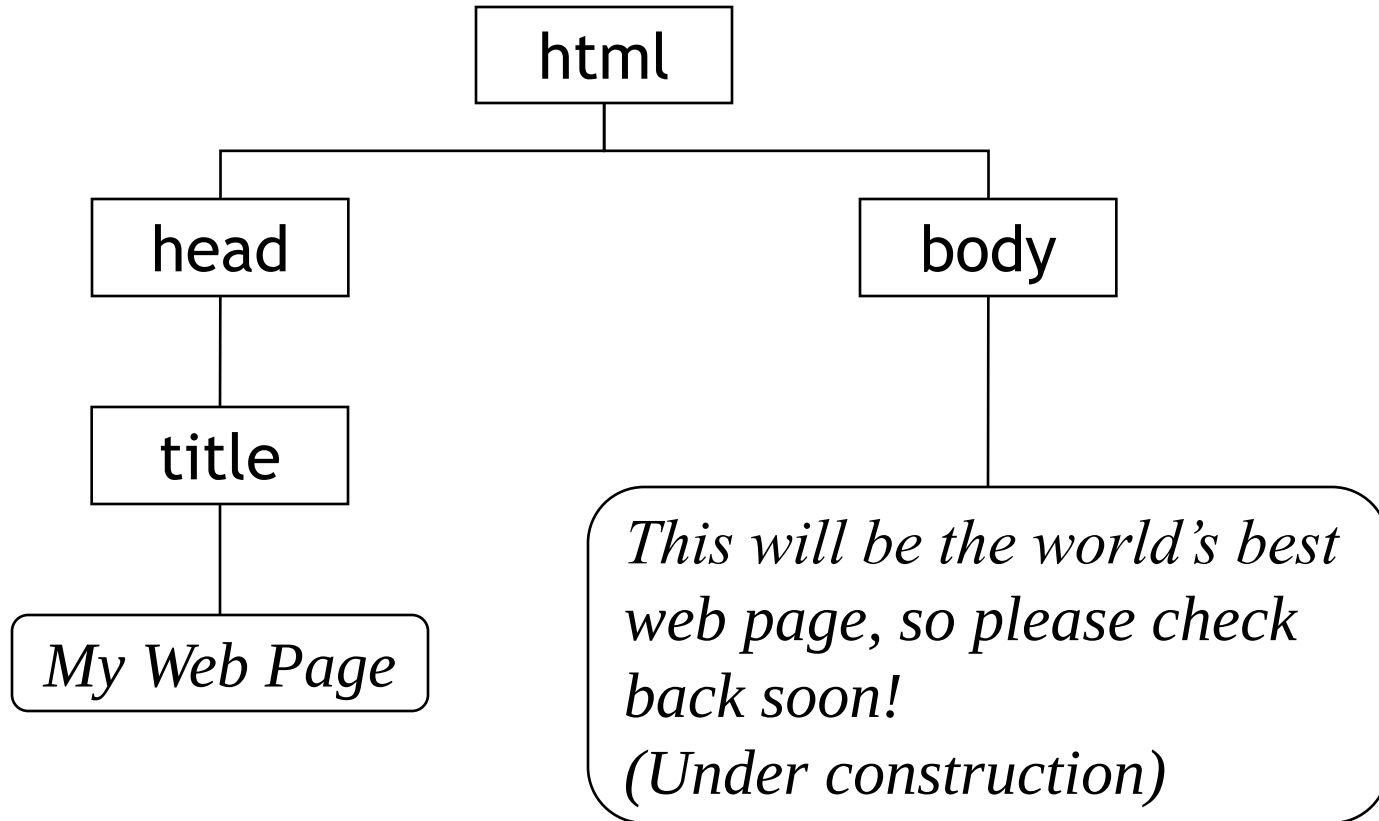
Structure of an HTML document

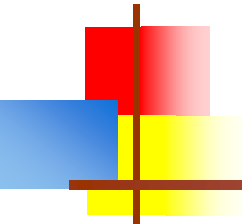
- An HTML document is contained within `<html>` tags
 - It consists of a `<head>` and a `<body>`, in that order
 - The `<head>` typically contains a `<title>`, which is used as the title of the browser window
 - Almost all other content goes in the `<body>`
- Hence, a fairly minimal HTML document looks like this:

```
<html>
  <head>
    <title>My Title</title>
  </head>
  <body>
    Hello, World!
  </body>
</html>
```



HTML documents are trees





Anatomy of an HTML Element

■ Element

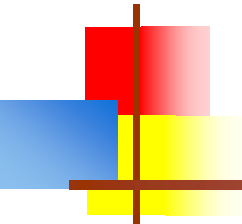
- Building blocks of web pages - an individual component of HTML
- Examples: paragraph, heading, table, list, div, link, image, etc.

■ Tag

- Opening tag marks the beginning of an element & closing tag marks the end
- Tags contain characters that indicate the tag's purpose

<tagname>Stuff in the middle**</tagname>**

<p>This is a sample paragraph.**</p>**



Anatomy of an HTML Element

■ Container Element

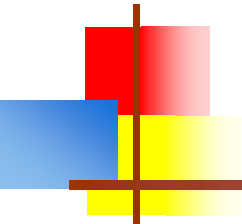
- An element that can contain other elements or content
- A paragraph (`<p>`) contains text

■ Stand Alone Element

- An element that cannot contain anything else

`<br />`

`<img />`



Anatomy of an HTML Element

■ Attribute

- Provides additional information about the HTML element
- Placed inside an opening tag, before the right angle bracket.
- Examples: class, id, language, style, source
- Multiple attributes are separated by spaces

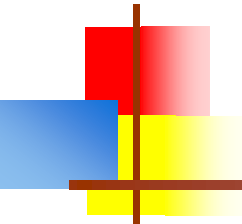
■ Value

- Value is the value assigned to a given attribute.
- Values must be contained inside quotation marks.

`<div id="copyright"> ©Girl Develop It logo 2016</div>`

``

`<a href="http://girldevelopit.com">Girl Develop It</a>`



Doctype

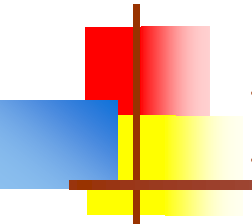
- The first thing on an HTML page is the doctype, which tells the browser which version of the markup language the page is using.

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML  
4.01 Transitional//EN" "http://  
www.w3.org/TR/html4/loose.dtd">
```

```
<!DOCTYPE html>
```

- * The doctype is case-insensitive.

DOCTYPE, doctype, DocType and DoCtYe are all valid.



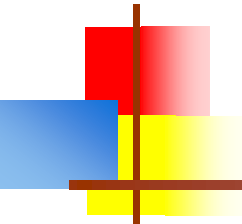
HTML Tag

- After <doctype>, the page content must be contained between <html> tags.

<!DOCTYPE html>

<html>

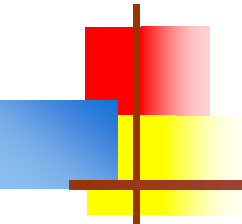
</html>



Head and Body Tags

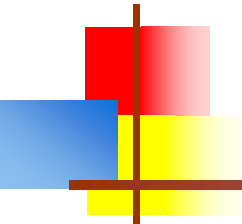
- **Head:** The head contains the title of the page & meta information about the page. Meta information is not visible to the user, but has many purposes, like providing information to search engines.
- **Body:** The body contains the actual content of the page. Everything that is contained in the body is visible to the user.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Title of the page </title>
  </head>
  <body>
    The page content here.
  </body>
</html>
```

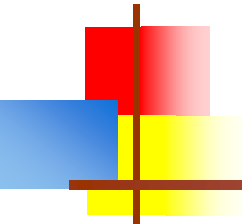
Whitespace

- **Whitespace** is any non-printing characters (space, tab, newline, and a few others)
- HTML treats all whitespace as word separators, and automatically *flows* text from one line to the next, depending on the width of the page
- To group text into paragraphs, with a blank line between paragraphs, enclose each paragraph in `<p>` and `</p>` tags
- To force HTML to use whitespace exactly as you wrote it, enclose your text in `<pre>` and `</pre>` tags (“pre” stands for “preformatted”)
 - `<pre>` also uses a monospace font
 - `<pre>` is handy for displaying programs



Nesting

- All elements "nest" inside one another
- Nesting is what happens when you put other containing tags inside other containing tags.
- For example, you would put the `<p>` inside of the `<body>` tags. The `<p>` is now nested inside the `<body>`
- **Nesting Order**
 - Whichever element OPENS first CLOSES last



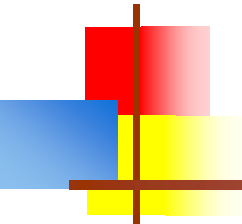
Element: Heading

- `<h1>Heading 1</h1>`
- `<h2>Heading 2</h2>`
- `<h3>Heading 3</h3>`
- `<h4>Heading 4</h4>`
- `<h5>Heading 5</h5>`
- `<h6>Heading 6</h6>`



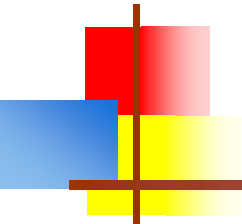
Heading 1
Heading 2
Heading 3
Heading 4
Heading 5
Heading 6

- * Heading number indicates hierarchy, not size. Think of outlines from high school papers



Links

- To link to another page, enclose the link text in `<a href="URL">` to `</a>`
 - Example: My Home page `<a href = http://www.google.co.in>Google India</a>` .
 - Link text will automatically be underlined and blue (or purple if recently visited)
 - This is an Anchor tag
- To link to another part of the same page,
 - Insert a **named anchor**: `<a name="refs">References</a>`
 - And link to it with: `<a href="#refs">My references</a>`
- To link to a named anchor from a different page, use `<a href="PageURL#refs">My references</a>`



Examples of Anchor tags

- Anchor tags with URLs that are relative to the current directory:

`<a href="reference.html"> The ref 1</a>`

`<a href="email/reference.html">The ref 2</a>`

- Anchor tags with relative URLs that navigate up the directory structure:

`<a href="../ref.html">Go back one directory level </a>`

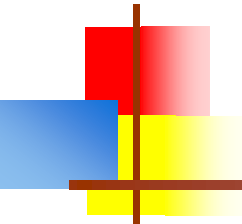
`<a href="../../ref.html">Go back 2 directory levels </a>`

- Anchor tags with URLs that are relative to the webapps directory:

`<a href="/jsps/ref.html">context root directory of web app</a>`

- Anchor tags with absolute URLs:

`<a href="http://www.irctc.com/email">An internet address</a>`



Link Attributes

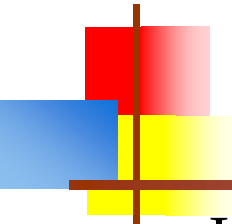
- Links can have attributes that tell the link to do different actions like open in a new tab, or launch your e-mail program.

`<a href="home.html" target="_blank">Link Text</a>`

- Link opens in a new window/tab with `target="_blank"`

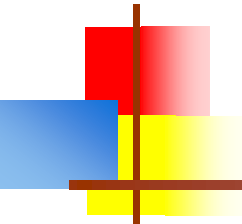
`<a href="mailto:sponsorships@girldevelopit.com">E-mail us!</a>`

- Adding `mailto:` directly before the email address means the link will open in the default email program.



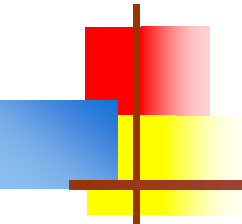
Images

- Images (pictures) are not part of an HTML page; the HTML just tells where to find the image
- To add an image to a page, use:
``
 - The `src` attribute is required; the others are optional
 - Attributes may be in any order
 - The `URL` may refer to any `.gif`, `.jpg`, or `.png` file
 - Other graphic formats are not recognized
 - The `alt` attribute provides a text representation of the image if the actual image is not downloaded
 - The `height` and `width` attributes, if included, will improve the display as the page is being downloaded
 - If `height` or `width` is incorrect, the image will be distorted
 - There is no `</img>` end tag, because `<img>` is *not* a container



Relative vs. Absolute Paths for Links & Images

- **Relative:** paths change depending on the page the link is on.
 - Links within the same directory need no path information.
"filename.jpg"
 - Subdirectories are listed without preceding slashes.
"img/filename.jpg"
- **Absolute:** paths refer to a specific location of a file, including the domain.
 - Typically used when pointing to a link that is not within your own domain.
 - Example: <https://www.sssihl.edu.in/chapters/html>



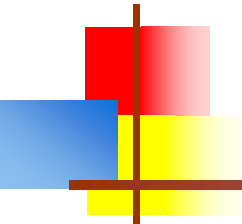
Lists

- Two of the kinds of lists in HTML are ordered, `<ol>` to `</ol>`, and unordered, `<ul>` to `</ul>`
- Ordered lists typically use numbers: 1, 2, 3, ...
- Unordered lists typically use bullets (•)
- The elements of a list (either kind) are surrounded by `<li>` and `</li>`

- Example:

The four main food groups are:

```
<ul>
  <li>Sugar</li>
  <li>Chips</li>
  <li>Caffeine</li>
  <li>Chocolate</li>
</ul>
```



Unordered and Ordered Lists

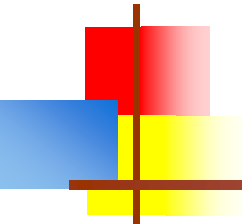
- `<ul>`
- `<li>List Item</li>`
- `<li>Another List Item</li>`
- `</ul>`
- `<ol>`
- `<li>List Item</li>`
- `<li>Another List Item</li>`
- `</ol>`

Unordered list (bullets)

- List Item
- Another List Item

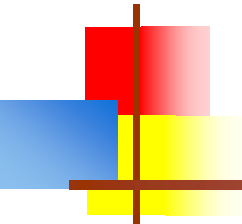
Ordered list (sequence)

1. List Item
2. Another List Item



Lists

- Example: To have an ordered list with letters A, B, C, ... instead of numbers, use `<ol type="A">` to `</ol>`
 - For lowercase letters, use `type="a"`
 - For Roman numerals, use `type="I"`
 - For lowercase Roman numerals, use `type="i"`
 - In this example, `type` is an attribute
 - JavaScript can most easily refer to HTML elements that use a `name` or `id` attribute
- Attributes are coded in starting tag only; multiple attributes are separated by spaces

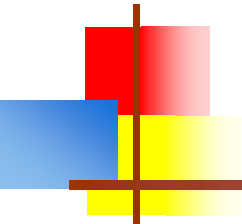


Comments

- You can add comments to your code that will not be seen by the browser, but only visible when viewing the code.

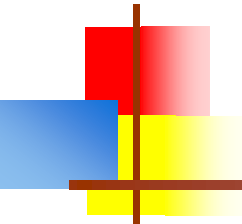
`<!-- Comment goes here -->`

- Comments can be used to organize your code into sections so you (or someone else) can easily understand your code. It can also be used to 'comment out' large chunks of code to hide it from the browser.



Entities

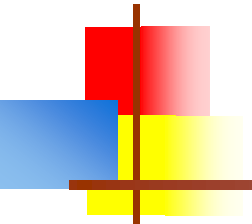
- Certain characters, such as `<`, have special meaning in HTML
- To put these characters into HTML without any special meaning, we have to use **entities**
- Here are some of the most common entities:
 - `<` represents `<`
 - `>` represents `>`
 - `&` represents `&`
 - `'` represents `'`
 - `"` represents `"`
 - ` ` represents a “nonbreaking space”--one that HTML does *not* treat as whitespace



Tables

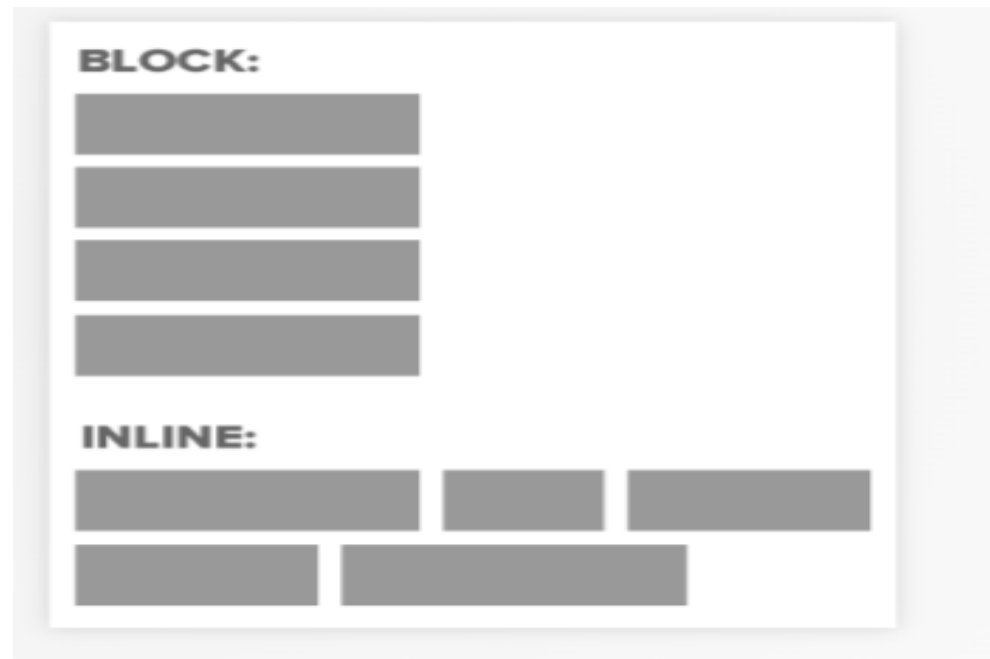
```
<table cellpadding="5" cellspacing="5" border="1">
<tr>
  <th>Product</th>
  <th>Price</th>
</tr>
<tr><td>Java Book</td><td>$22</td></tr>
<tr><td>JSP and Servlet Book</td><td>$28</td></tr>
</table>
```

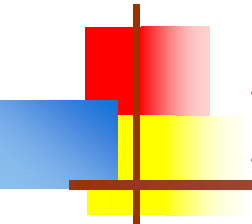
Product	Price
Java Book	\$22
JSP and Servlet Book	\$28



Inline vs Block

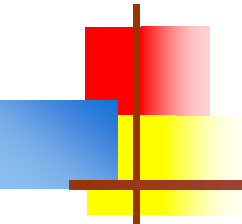
- So far, we have mostly seen "**block**" elements. They appear on the next line, like paragraphs.
- There are also "**inline**" elements. They appear on the same line that they are written on.
- example of inline and block elements





Block & Inline Elements

- CSS divides HTML into two types: inline and block.
- After block elements, browsers render a new line.
- **Inline elements:** img, a, br, em, strong
- **Block elements:** p, h1, ul, li, almost everything else



Element: div

- Block level element. Each new **div** is rendered on a new line.
- A division, or section of content within an HTML page.
- Used to group elements to format them with CSS.
- Apply IDs and Classes to **divs** to control their styles with CSS.

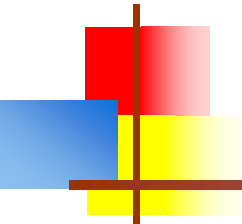
<div>

<p>Content<p>

<p>Content<p>

</div>

- You can wrap groups of elements in a div to style them differently.

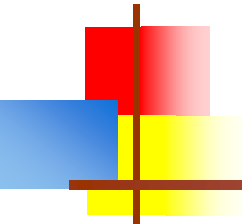


Element: span

- Inline element. Each new span is rendered next to each other & only wraps when it reaches the edge of the containing element.
- Can be used to apply styles to text inline so as not to break the flow of content.

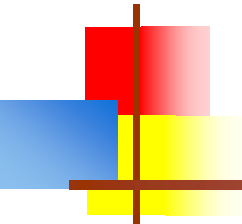
`<p>Paragraph with <span style="color:teal;">teal</span> text.</p>`

Paragraph with teal text.



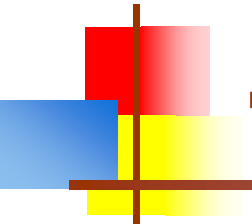
Forms

- `<form>` is just another kind of HTML tag
- HTML forms are used to create (rather primitive) GUIs on Web pages
 - Usually the purpose is to ask the user for information
 - The information is then sent back to the server
- A **form** is an area that can contain **form elements**
 - The syntax is: `<form parameters> ...form elements... </form>`
 - Form elements include: buttons, checkboxes, text fields, radio buttons, drop-down menus, etc
 - Other kinds of HTML tags can be mixed in with the form elements
 - A form usually contains a **Submit** button to send the information in the form elements to the server
 - The form's *parameters* tell JavaScript how to send the information to the server (there are two different ways it could be sent)
 - Forms can be used for other things, such as a GUI for simple programs



The <form> tag

- The <form *arguments*> ... </form> tag encloses form elements (and probably other HTML as well)
- The arguments to form tell what to do with the user input
 - `action="url"` (required)
 - Specifies where to send the data when the `Submit` button is clicked
 - `method="get"` (default)
 - Form data is sent as a URL with `?form_data` info appended to the end
 - Can be used *only* if data is all ASCII and not more than 100 characters
 - `method="post"`
 - Form data is sent in the body of the URL request
 - Cannot be bookmarked by most browsers
 - `target="target"`
 - Tells where to open the page sent as a result of the request
 - `target= _blank` means open in a new window
 - `target= _top` means use the same window



The `<input>` tag

- Most, but not all, form elements use the `input` tag, with a `type="..."` argument to tell which kind of element it is
 - `type` can be `text`, `checkbox`, `radio`, `password`, `hidden`, `submit`, `reset`, `button`, `file`, or `image`
- Other common `input` tag arguments include:
 - `name`: the name of the element
 - `value`: the “value” of the element; used in different ways for different values of `type`
 - `readonly`: the value cannot be changed
 - `disabled`: the user can’t do anything with this element
 - Other arguments are defined for the `input` tag but have meaning only for certain values of `type`

Text input

A text field:

```
<input type="text" name="textfield" value="with an initial value">
```

A text field:

A multi-line text field

```
<textarea name="textarea" cols="24" rows="2">Hello</textarea>
```

A multi-line text field:

A password field:

```
<input type="password" name="textfield3" value="secret">
```

A password field:

- Note that two of these use the **input** tag, but one uses **textarea**

Buttons

- A submit button:
`<input type="submit" name="Submit" value="Submit">`
- A reset button:
`<input type="reset" name="Submit2" value="Reset">`
- A plain button:
`<input type="button" name="Submit3" value="Push Me">`

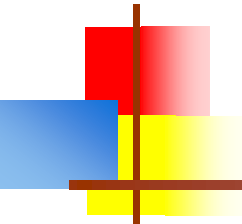
A submit button: 

A reset button: 

A plain button: 

- **submit**: send data
- **reset**: restore all form elements to their initial state
- **button**: take some action as specified by JavaScript

- Note that the type is **input**, not “button”

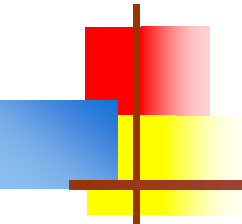


Checkboxes

- A checkbox:
`<input type="checkbox" name="checkbox"
value="checkbox" checked>`

A checkbox: ☒

- `type: "checkbox"`
- `name`: used to reference this form element from JavaScript
- `value`: value to be returned when element is checked
- Note that there is *no text* associated with the checkbox—you have to supply text in the surrounding HTML



Radio buttons

Radio buttons:


```
<input type="radio" name="radiobutton" value="myValue1">  
male<br>
```

```
<input type="radio" name="radiobutton" value="myValue2" checked>  
female
```

Radio buttons:

☐ male
☒ female

- If two or more radio buttons have the same **name**, the user can only select one of them at a time
 - This is how you make a radio button “group”
- If you ask for the value of that **name**, you will get the **value** specified for the selected radio button
- As with checkboxes, radio buttons do not contain any text

Drop-down menu or list

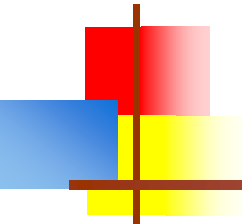
- A menu or list:

```
<select name="select">  
  <option value="red">red</option>  
  <option value="green">green</option>  
  <option value="BLUE">blue</option>  
</select>
```

A menu or list: 

- Additional arguments:

- **size**: the number of items visible in the list (default is "1")
- **multiple**: if set to "true", any number of items may be selected (default is "false")

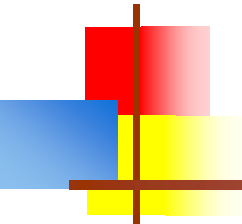


Hidden fields

- `<input type="hidden" name="hiddenField" value="nyah">`
 <-- right there, don't you see it?

A hidden field: <-- right there, don't you see it?

- What good is this?
 - All **input** fields are sent back to the server, including hidden fields
 - This is a way to include information that the user doesn't need to see (or that you don't want her to see)
 - The **value** of a hidden field can be set programmatically (by JavaScript) before the form is submitted



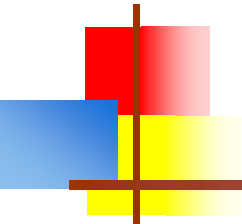
A complete example

```
<html>
<head>
<title>Get Identity</title>
<meta http-equiv="Content-Type" content="text/html;
      charset=iso-8859-1">
</head>
<body>
<p><b>Who are you?</b></p>
<form method="post" action="">
  <p>Name:
    <input type="text" name="textfield">
  </p>
  <p>Gender:
    <input type="radio" name="gender" value="m">Male
    <input type="radio" name="gender" value="f">Female</p>
</form>
</body>
</html>
```

Who are you?

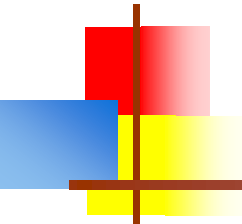
Name:

Gender: ☐ Male ☐ Female



The rest of HTML

- HTML is a large markup language, with a lot of options
 - If you aren't already familiar with it, you should study one or more of the tutorials (there are many online)
 - Your browser's **View -> Source** command is a great way to see how things are done in HTML
 - HTML pages often contain JavaScript code
 - There is no such “thing” as DHTML (Dynamic HTML)
 - DHTML is simply HTML with several other technologies mixed in, such as JavaScript



HTML5

- HTML5 offers new elements for better document structure and semantics
- Some of the most commonly used new tags include:

`<header></header>`

`<nav></nav>`

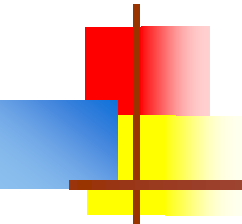
`<article></article>`

`<section></section>`

`<main></main>`

`<footer></footer>`

- A full list can be found [here](#).



The End
